

Establishing Governance and Writing a Clarified Specification

Set the standard everything else gets checked against, then write against it.

What You'll Learn

- 1 Encode project or per-service governance in a constitution
- 2 Write a spec from both a feature request and a defect report using EARS-style requirements
- 3 Run a clarify pass to surface ambiguity before planning
- 4 Map the SDD lifecycle onto your team's existing SDLC

The Constitution

Encodes principles, code quality standards, testing requirements, and architectural constraints, so "good" is defined once instead of re-argued on every pull request.

Written once up front, informed by Module 2's discovery, and checked against by every later phase, which is what keeps plans and code from drifting away from team standards over time.

Without it, every generated plan and every piece of code is evaluated against nothing, which means quality depends entirely on whoever happens to be reviewing that day.

EARS-Style Requirements and Clarify

EARS format: "When/if [condition]... the system shall [response]." Testable, unambiguous, one behavior per statement, so a requirement can be checked off as done or not done, with no room for interpretation.

Specs come from two different sources on this course: a new-feature request and an existing defect pulled from the tracker, because those two inputs need to produce the same rigor of spec even though they start from very different starting points.

A clarify pass surfaces ambiguities and edge cases before they get baked into a plan, which is far cheaper than discovering them after code has already been written against the wrong assumption.

The SDD lifecycle isn't a separate process: it maps onto the SDLC you already run.

Constitution and clarified spec approximate requirements and design; plan and tasks approximate technical design and iteration planning, so nothing here replaces your existing process, it just gives each phase a concrete, agent-readable artifact.

Checklist and analyze approximate QA and test planning, implement approximates development, and review plus living docs approximate code review, QA, and maintenance, which is why teams can adopt SDD without abandoning the SDLC stages they already know.

APPLYING THE CONCEPTS

GitHub Spec Kit

Constitution, Specify, Clarify

`/speckit.constitution`

Creates or updates the constitution and keeps dependent templates in sync, so a change to a principle automatically propagates instead of quietly going stale in one place.

`/speckit.specify`

Creates the feature spec from natural language, focused on what and why, not tech stack, which keeps the spec readable by non-engineers and stable even if the implementation approach later changes.

`/speckit.clarify`

Asks up to five targeted questions about underspecified areas and encodes your answers back into spec.md. Run it as many times as needed, since one pass often surfaces a follow-up question the first round didn't anticipate.

APPLYING THE CONCEPTS

OpenSpec

Config Files, Deltas, and Explore

Governance gap: No dedicated governance command: project conventions live in openspec/config.yaml rather than a generated document, so governance is configuration you maintain directly instead of an artifact a command produces for you.

`/opsx:propose`

Generates proposal.md plus delta specs/ marking ADDED, MODIFIED, REMOVED requirements, each with Given/When/Then scenarios: OpenSpec's equivalent of testable acceptance criteria, playing the same role EARS statements play in Spec Kit.

Clarify gap: No single dedicated clarify command: ambiguity gets sharpened beforehand with /opsx:explore, or reconciled afterward with /opsx:update, so the clarify step is distributed across the workflow rather than being one named command.

Module 3 at a Glance

	GitHub Spec Kit	OpenSpec
Governance	/speckit.constitution (dedicated command)	openspec/config.yaml (no generated document)
Spec writing	/speckit.specify	/opsx:propose (delta specs, Given/When/Then)
Clarify	/speckit.clarify (dedicated command)	/opsx:explore before, /opsx:update after

Key Takeaways

- Governance has to exist before code is judged against it, otherwise every review starts from a different, unstated standard.
- EARS keeps requirements testable regardless of which framework carries them, since the format itself, not the tool, is what makes a requirement checkable.
- Clarify early: it's cheap now and expensive once a plan is built on top of ambiguity that nobody caught in time.